

Reviewer Pack — Negative SoS-Fourier Hankel Spectrum Separates Sipser from A...

Entry c1588ea452b8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 19:25:20 UTC

Negative SoS-Fourier Hankel Spectrum Separates Sipser from ACC0[3]

Entry ID: c1588ea452b8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 19:25:20 UTC

1. Conjecture

Field A (mathematical branch): Sum-of-Squares hierarchy pseudo-moment matrices built from Walsh-Hadamard (Fourier-Hankel) signatures **Field B** (complexity object): ACC0[3] circuit complexity of the depth-2 Sipser function S_n (top OR of bottom ANDs of fan-in $\sim \sqrt{n}$)

Statement:

For $f: \{0,1\}^n \rightarrow \{-1,+1\}$, let $B_d = \{S \subset [n] : |S| \leq d\}$ and let $Q_d(f)$ be the $|B_d| \times |B_d|$ symmetric matrix with entries $Q_d(f)[S,T] = \widehat{f}(S \oplus T)$, where $\widehat{f}(U) = 2^{-n} \sum_x f(x) (-1)^{\langle U, x \rangle}$. Define the SoS-Fourier negativity gap $NG_d(f) := -\lambda_{\min}(Q_d(f))$. The conjecture: (i) for every f computable by a depth-3 ACC0[3] circuit of size s , $NG_d(f) \leq s^{-1/2}$ for all $d \leq \log s$, and (ii) for the explicit Sipser function S_n with bottom fan-in $\lfloor \sqrt{n} \rfloor$ and top fan-in $\lceil \sqrt{n} \rceil$, $NG_2(S_n) \geq 1/(8n)$. A single ACC0[3] circuit (with size up to $n \log n$, depth 3) achieving $NG_2 \geq 1/(8n)$, or a single n in $\{6,8,10\}$ where Sipser yields $NG_2 < 1/(8n)$, refutes it.

Rationale (proposer's reasoning):

$Q_d(f)$ is the Gram matrix of the SoS pseudo-expectation $E[\chi_S \cdot \chi_T \cdot f]$; its negative-spectrum component encodes obstructions to writing f as a sum-of-squares of degree- d Fourier polynomials. Random and ACC0[3] truth tables have Fourier mass concentrated at low $|U|$, forcing Q_d to be near-PSD (small NG). Sipser's tight DNF/CNF gap creates a sign-cancellation pattern on medium-sized Fourier coefficients that, viewed through the Hankel skew, should produce a robust negative eigenvalue at degree 2 - and crucially the property 'high negativity' is SMALL (random f gives $NG = O(2^{-n/2})$), so the property avoids the Razborov-Rudich largeness condition while remaining useful and subexp-computable.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 21fe76326b885476

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each n in $\{6, 8, 10\}$, compute $NG_2(S_n)$ and NG_2 for 30 random ACC0[3] circuits (fixed seeds 0-29). Support requires $NG_2(S_n) \geq 1/(8n)$ AND $NG_2(S_n) - \max_{\text{seed}} NG_2(\text{circuit}) \geq 1/(8n)$ for all three n . Any violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.93	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Sum-of-Squares Fourier Hankel pseudo-moment Boolean function lower bound - ACC0 circuit lower bound Walsh-Hadamard spectrum Sipser depth-3 - SoS hierarchy pseudo-expectation Fourier coefficients minimum eigenvalue Boolean

Top relevant hits considered: - [<http://arxiv.org/abs/2511.07739v3>] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/2406.18700v4>] Structure of sparse Boolean functions over Abelian groups, and its application to testing - [<http://arxiv.org/abs/2504.19966v3>] Quantum circuit lower bounds in the magic hierarchy - [<http://arxiv.org/abs/1508.06336v1>] SPRIGHT: A Fast and Robust Framework for Sparse Walsh-Hadamard Transform - [<http://arxiv.org/abs/0906.069>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/2004.14244v3>] Eulerianity of Fourier coefficients of automorphic forms - [<http://arxiv.org/abs/1410.7075v1>] A note on the Vilenkin-Fourier coefficients

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
from itertools import combinations

def fourier_hat(f, U):
    n = len(U)
    return sum(f(x) * (-1)**sum(1 for i in range(n) if x & (1 << i)) / 2**n for x in range(2**n))

def gram_matrix(f, d):
    B_d = list(combinations(range(len(f)), d))
    Q_d = [[0] * len(B_d) for _ in range(len(B_d))]
    for S, T in combinations(B_d, 2):
        Q_d[B_d.index(S)][B_d.index(T)] = fourier_hat(f, S ^ T)
        Q_d[B_d.index(T)][B_d.index(S)] = Q_d[B_d.index(S)][B_d.index(T)]
```

```

Q_d[B_d.index(B_d[0])][B_d.index(B_d[0])] = sum(fourier_hat(f, U)**2 for U in B_d) / len(B_d)
return Q_d

def eigenvalues(Q_d):
    n = len(Q_d)
    if n == 1:
        return [Q_d[0][0]]

    # Gaussian elimination to find the eigenvalues
    A = [[Q_d[i][j] for j in range(n)] for i in range(n)]
    for k in range(n):
        max_row = max(range(k, n), key=lambda i: abs(A[i][k]))
        A[k], A[max_row] = A[max_row], A[k]
        for j in range(k + 1, n):
            factor = A[j][k] / A[k][k]
            for m in range(n):
                A[j][m] -= factor * A[k][m]

    # Extract the eigenvalues from the diagonal
    return [A[i][i] for i in range(n)]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [6, 8, 10]
    results = []

    for n in n_values:
        f = lambda x: (-1)**sum(1 for i in range(n) if x & (1 << i))
        S_n = set(range(n))

        Q_2_S_n = gram_matrix(f, 2)
        NG_2_S_n = -min(eigenvalues(Q_2_S_n))

        max_NG_2_circuit = 0
        for _ in range(30):
            circuit = [random.choice(['AND', 'OR', 'MOD3']) for _ in range(n)]
            NG_2_circuit = -min(eigenvalues(gram_matrix(circuit, 2)))
            if NG_2_circuit > max_NG_2_circuit:
                max_NG_2_circuit = NG_2_circuit

        results.append({
            "metric_name": "NG_2",
            "metric_value": NG_2_S_n,
            "instances_tested": 1,
            "conjecture_holds": NG_2_S_n >= Fraction(1, 8 * n) and NG_2_S_n - max_NG_2_circuit >= 0,
            "counterexample": "" if NG_2_S_n >= Fraction(1, 8 * n) and NG_2_S_n - max_NG_2_circuit >= 0 else max_NG_2_circuit
        })

    return {
        "seed": seed,
        "metric_name": "NG_2",
        "metric_value": sum(result["metric_value"] for result in results),
        "instances_tested": len(results),
        "conjecture_holds": all(result["conjecture_holds"] for result in results),
        "counterexample": "" if all(result["conjecture_holds"] for result in results) else next(re
    }

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(3, 6)]

    for seed in seeds:
        print(f"TRIAL: {run_trial(seed)}")

    NG_2_values = [result["metric_value"] for result in run_trial(seeds[0])["instances"]]
    support_fraction = sum(result["conjecture_holds"] for result in run_trial(seeds[0])["instances"])

    if all(result["conjecture_holds"] for result in run_trial(seeds[0])["instances"]):
        print(f"RESULT: SUPPORTED mean={sum(NG_2_values)/len(NG_2_values):.6f} std={(sum((x - sum(NG_2_values)/len(NG_2_values))**2 for x in NG_2_values))**.5f}")
    elif any(not result["conjecture_holds"] for result in run_trial(seeds[0])["instances"]):
        first_failing_seed = next(seed for seed, result in enumerate(run_trial(seeds[0])["instances"]) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"NG_2(S_n) < 1/(8n)\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3f4e32f9.py", line 89, in <module>
    print(f"TRIAL: {run_trial(seed)}")
                        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3f4e32f9.py", line 57, in run_trial
    Q_2_S_n = gram_matrix(f, 2)
                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3f4e32f9.py", line 22, in gram_matrix
    B_d = list(combinations(range(len(f)), d))
                                ~~~~~^
TypeError: object of type 'function' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` (`len()` called on a function) before any `NG_2` values were computed, so the pre-registered support condition could not be evaluated. | next: Fix the `gram_matrix` call to use `n` (the input length) instead of `len(f)`, then rerun the trials for `n` in `{6,8,10}`.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	183246
2	preregistration	claude_max	opus	0	0	5443
3	novelty	claude_max	opus	0	0	4307
4	novelty	claude_max	opus	0	0	7895
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14933
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14130
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	150893
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14777
9	judge	claude_max	opus	0	0	5077

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 400702 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/c1588ea452b8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c1588ea452b8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c1588ea452b8.tar.gz` (if generated)